Sistema de Gestión de Pedidos en Línea para Restaurante

Implementación de MongoDB y persistencia políglota

Informe técnico de la entrega

Proyecto Mesa Roja - Vue 3, Flask, PostgreSQL y MongoDB

Documento elaborado a partir del código de esta entrega. Las pruebas de integración con servicios reales se indican como pendientes cuando requieren credenciales o infraestructura externa.

1. Introducción

Este proyecto permite consultar un menú, registrar clientes, crear pedidos y hacer seguimiento a sus entregas. La aplicación tiene una interfaz Vue y una API Flask.

El objetivo de esta entrega es complementar la persistencia relacional con MongoDB para guardar eventos y actividades que pueden tener datos adicionales. No se reemplazó el frontend ni se creó otra aplicación: se extendió el proyecto Mesa Roja.

2. Problemática y punto de partida

La versión local que se revisó no tenía conexión ni tablas de PostgreSQL: los usuarios y pedidos vivían en listas Python y se perdían al detener Flask. Por eso, antes de agregar MongoDB fue necesario incorporar la persistencia relacional que requiere el enunciado, conservando los endpoints y las validaciones del flujo de compra.

No se migró información previa de PostgreSQL porque esa instalación no existía en el código examinado.

3. Datos estructurados y datos dinámicos

Los usuarios, productos, pedidos, detalles, precios y existencias tienen campos y relaciones estables. PostgreSQL los organiza en tablas y permite validar referencias e inventario en transacciones.

Los eventos de actividad, cambios de estado, acciones administrativas y datos contextuales varían según la operación. MongoDB permite conservarlos como documentos con un campo datos flexible, sin alterar el esquema SQL cada vez que se agrega contexto.

4. Justificación de MongoDB y colecciones

MongoDB se usa para consultar y conservar eventos por pedido y fecha. PyMongo inserta documentos JSON/BSON y crea índices sobre pedido_id y fecha.

Colección	Contenido guardado
actividades_usuarios	usuario_id, pedido_id, tipo, descripcion, fecha, responsable y datos
historial_pedidos	pedido_id, estados anterior y nuevo, usuario, tipo, fecha y datos
registros_sistema	acción, usuario, descripción, fecha y datos de auditoría

5. Comparación de responsabilidades

PostgreSQL es la fuente principal para el estado actual del negocio: inventario, clientes, pedidos y ventas. MongoDB es la fuente de consulta para la actividad histórica y los registros flexibles. Los identificadores numéricos PostgreSQL se guardan como pedido_id y usuario_id en los documentos; el ObjectId identifica cada actividad.

6. Persistencia políglota e integridad

No existe una transacción distribuida entre PostgreSQL y MongoDB. Para no confirmar un pedido y perder su evento si MongoDB falla, la transacción SQL guarda el pedido, descuenta el inventario y añade los eventos a cola_eventos en un único commit. Luego Flask publica cada evento en MongoDB.

La cola usa un UUID como identificador de documento y MongoDB lo guarda como _id. Si un reintento encuentra ese documento, la inserción duplicada se trata de forma idempotente y se marca el evento como publicado. Si la conexión falla, se guarda el error y el evento permanece disponible para reintento.

7. Operaciones CRUD y conexión

PostgresStore crea el esquema desde backend/schema.sql, inicializa el catálogo y realiza las transacciones de usuarios, pedidos y estados. MongoStore crea índices y expone las operaciones para actividades. La aplicación necesita DATABASE_URL y MONGODB_URI; MONGODB_DATABASE es opcional y su valor predeterminado es mesa_roja.

POST /actividades registra una actividad. GET /actividades permite listar y filtrar. GET /actividades/<id> consulta una actividad. PATCH /actividades/<id> actualiza campos permitidos y DELETE /actividades/<id> elimina el documento. Crear, modificar o eliminar actividades genera además un registro en registros_sistema.

8. Consulta combinada PostgreSQL y MongoDB

GET /pedidos/<id>/historial obtiene primero el pedido, cliente y productos desde PostgreSQL. Luego publica eventos pendientes de ese pedido y consulta en MongoDB historial_pedidos, actividades_usuarios y registros_sistema. La respuesta reúne las cuatro propiedades bajo un único JSON.

Si no se pueden publicar eventos del pedido, el endpoint devuelve un error 503 claro en lugar de ocultar que el historial todavía está incompleto.

```
{"pedido": {...}, "historial": [...], "actividades": [...], "registros_sistema": [...]}
```

9. Integración con la interfaz y endpoints

La pantalla de seguimiento obtiene pedidos persistidos, cambia sus estados por PATCH y muestra el historial combinado para pedidos pendientes y entregados. La compra existente continúa registrando usuario y pedido con los mismos endpoints.

La API también conserva GET /productos, POST /usuarios, POST /pedidos, GET /pedidos/<id> y GET /reportes/ventas. Se añadieron GET /pedidos, PATCH /pedidos/<id>/estado, las rutas CRUD de /actividades, GET /pedidos/<id>/historial, GET /salud y POST /eventos/pendientes/reintentar.

- GET /pedidos/<id>/historial

- POST /actividades

- GET /actividades?pedido_id=<id>

- PATCH /pedidos/<id>/estado

10. Pruebas y evidencias

La suite local contiene nueve pruebas automatizadas con adaptadores falsos: verifica rutas, validaciones, CRUD, respuesta combinada, configuración y reintentos idempotentes. El build de producción de Vite también se ejecutó correctamente.

Estas pruebas no son una conexión real a bases de datos. En el entorno de esta entrega no había DATABASE_URL ni MONGODB_URI y los puertos locales 5432 y 27017 no estaban disponibles. Por ello, no se afirma que el CRUD o la conexión externa hayan sido verificados con servicios reales.

backend/integration_smoke.py contiene una prueba real reproducible: con ambas bases configuradas crea datos temporales, comprueba los endpoints y limpia el pedido y los documentos que genera. Su ejecución requiere completar .env con credenciales propias.

11. Seguridad y configuración

Las credenciales se leen desde variables de entorno y .env está ignorado por Git. El archivo .env.example solo contiene valores de ejemplo.

La aplicación conserva un login de demostración en frontend y la API no implementa autenticación ni autorización de servidor. Por tanto, las rutas de actividades no deben exponerse a Internet ni utilizarse con datos reales hasta agregar control de acceso.

12. Conclusiones

La solución separa datos de negocio y eventos: PostgreSQL protege transacciones y stock, mientras MongoDB permite conservar una bitácora flexible. La cola de eventos hace explícito el límite entre las dos bases y evita perder la trazabilidad por una interrupción temporal.

El código y la documentación incluyen instrucciones para reproducir la configuración. La verificación final contra PostgreSQL y MongoDB reales queda pendiente de que se proporcionen y configuren las conexiones.